בס"ד. 24.04.26.

דו"ח מסכם: השוואת ביצועים – מנגנון חישוב נוסחאות דינמי.

1. מבוא ומתודולוגיית המדידה

בפרויקט זה עסקתי בפיתוח והשוואה של מנגנונים לחישוב נוסחאות דינמיות, במטרה להתמודד עם מערכות בהן הלוגיקה העסקית משתנה באופן תדיר ונדרש פתרון גמיש, יעיל וניתן להרחבה.
במסגרת העבודה מימשתי מספר גישות טכנולוגיות שונות, תוך בחינת ההשפעה של מיקום החישוב (מסד נתונים מול שכבת אפליקציה) על ביצועי המערכת.

במהלך מימוש הפרויקט, לאחר בחינה ראשונית של זמני הריצה הכוללים, זיהיתי כי בשיטות המבוססות על שפות חיצוניות (כגון #C ו־ Python), זמן הריצה אינו מורכב מרכיב אחד בלבד, אלא משני רכיבים עיקריים: 1. זמן החישוב , 2. זמן הכתיבה למסד הנתונים.

נמצא כי קיים פער משמעותי בין שני רכיבים אלו, הנובע בעיקר מהצורך בהעברת נתונים בין שכבות המערכת (מהמסד לאפליקציה ובחזרה), וכן מהעלות של פעולת הכתיבה למסד הנתונים.

לעומת זאת, בשיטת SQL , החישוב והכתיבה מתבצעים ישירות בתוך מסד הנתונים, ולכן לא ניתן להפריד בצורה מובהקת .

לאור תובנה זו, החלטתי להרחיב את מודל המדידה ולהוסיף שתי עמודות נוספות עבור השיטות שאינן מבוססות SQL :

- **זמן החישוב – (Calc Time)** הזמן הנדרש לביצוע החישוב עצמו.
- **זמן הכתיבה למסד הנתונים – (Write Time)** הזמן הנדרש לעדכון התוצאות במסד.
- בנוסף, נשמר גם **Run Time** , המייצג את זמן הריצה הכולל (חישוב + כתיבה).

הרחבה זו אפשרה לבצע ניתוח מעמיק ומדויק יותר של ביצועי כל שיטה, ולהבין בצורה ברורה מהו מקור העיכובים — האם בתהליך החישוב עצמו או בפעולות הקלט/פלט (I/O) .

2. שיטות המימוש

2.1 SQL Dynamic

בשיטה זו מימשתי את החישוב ישירות במסד הנתונים באמצעות SQL דינמי ופרוצדורה מאוחסנת.
הנוסחאות מופעלות על כלל הנתונים ללא מעבר בין שכבות (Zero Data Movement) .

במימוש נעשה שימוש ב־ Cursor על מנת לעבור על כל נוסחה בנפרד, דבר שאפשר מדידה מדויקת של זמן הריצה לכל נוסחה (Run Time) ורישום ללוג.

ניתן היה לממש פתרון יעיל יותר באמצעות גישה סטטית (Set-Based) ללא Cursor, אך בגישה זו לא ניתן למדוד זמן ריצה לכל נוסחה בנפרד. לכן נבחרה הגישה הנוכחית, המאפשרת השוואה מדויקת וניתוח ביצועים מפורט.

שיטה זו הציגה את הביצועים הטובים ביותר.

2.2 C# Bulk

בשיטה זו שלפתי את הנתונים מהמסד לאפליקציית #C , שם ביצעתי חישוב באמצעות DataTable.Compute .

לאחר מכן כתבתי את התוצאות חזרה למסד באמצעות SqlBulkCopy, מה ששיפר משמעותית את זמן הכתיבה.
זמן החישוב בשיטה זו גבוה יותר מ־ SQLמאחר והוא מתבצע מחוץ למסד הנתונים, אך זמן הכתיבה משתפר משמעותית בזכות שימוש ב־Bulk .

לכן, שיטה זו מספקת איזון טוב בין גמישות לביצועים, אך עדיין פחות יעילה מ־ SQL מבחינת זמן הריצה הכולל.

2.3 Python_eval

בשיטה זו שלפתי את הנתונים ל- Python וביצעתי חישוב לכל רשומה באמצעות eval המאפשר הרצת ביטויים בצורה גמישה.
את התוצאות כתבתי למסד הנתונים במנות (batch) .

מצאתי כי מדובר בשיטה גמישה, אך פחות יעילה, בעיקר בשל חישוב איטרטיבי והעלות ש eval מוסיף overhead נוסף.

2.4 Python_pandas_vec

בשיטה זו הנתונים נטענים ל־ DataFrame של pandas והחישוב מתבצע בצורה וקטורית (vectorization) על עמודות שלמות במקום חישוב שורה־שורה.

הנוסחאות מחושבות באמצעות eval עם פונקציות של NumPy , ובמקרה של תנאים נעשה שימוש ב־ np.where , מה שמאפשר ביצוע חישובים מהירים ויעילים על כל הנתונים בבת אחת.

לאחר החישוב, כל התוצאות מאוחדות ונכתבות למסד הנתונים במנות גדולות באמצעות executemany , תוך שימוש ב־ fast_executemany לשיפור ביצועי הכתיבה.

זמן החישוב בשיטה זו נמוך מאוד בזכות עבודה וקטורית, אך זמן הכתיבה למסד נשאר גבוה ומהווה את הגורם המרכזי בזמן הריצה הכולל.

לכן, השיטה מציגה ביצועי חישוב מצוינים, אך פחות יעילה כאשר נדרש עדכון מאסיבי של נתונים במסד.

3. השוואת ביצועים

מהבדיקות שביצעתי על מגוון נוסחאות (פשוטות ומורכבות), ראיתי כי שיטת SQL מספקת את הביצועים הטובים ביותר, מאחר והחישוב מתבצע ישירות בתוך מסד הנתונים ללא מעבר בין שכבות.
שיטת Pandas הציגה זמן חישוב מהיר מאוד, אך זמן הכתיבה למסד פגע בזמן הכולל.
שיטת #C סיפקה איזון טוב בין גמישות לביצועים, בעיקר בזכות שימוש ב- Bulk .
לעומת זאת, שיטת Python_eval הציגה את הביצועים הנמוכים ביותר עקב חישוב איטרטיבי.

לשם המחשה, סיכמתי את ההבדלים בין השיטות:

שיטה	חישוב	כתיבה	גמישות
SQL Dynamic	גבוהה מאוד	מקסימלית	בינונית
C# Bulk	בינונית	גבוהה	גבוהה
Python Pandas	גבוהה מאוד	בינונית	גבוהה מאוד
Python eval	נמוכ	בינונית	גבוהה

4. מסקנות

מהניתוח שביצעתי הסקתי כי ביצועי המערכת אינם תלויים רק באלגוריתם, אלא גם במיקום החישוב ובזרימת הנתונים בין שכבות.

כאשר הנתונים נמצאים במסד הנתונים –
הגעתי למסקנה כי הפתרון היעיל ביותר הוא SQL , מאחר והוא חוסך את עלויות העברת הנתונים ומאפשר חישוב אופטימלי.

כאשר נדרשת גמישות גבוהה יותר או לוגיקה מורכבת –
מצאתי כי ניתן להשתמש ב - Python ובעיקר (Pandas) או ב - C# תוך צמצום פעולות I/O ככל האפשר.

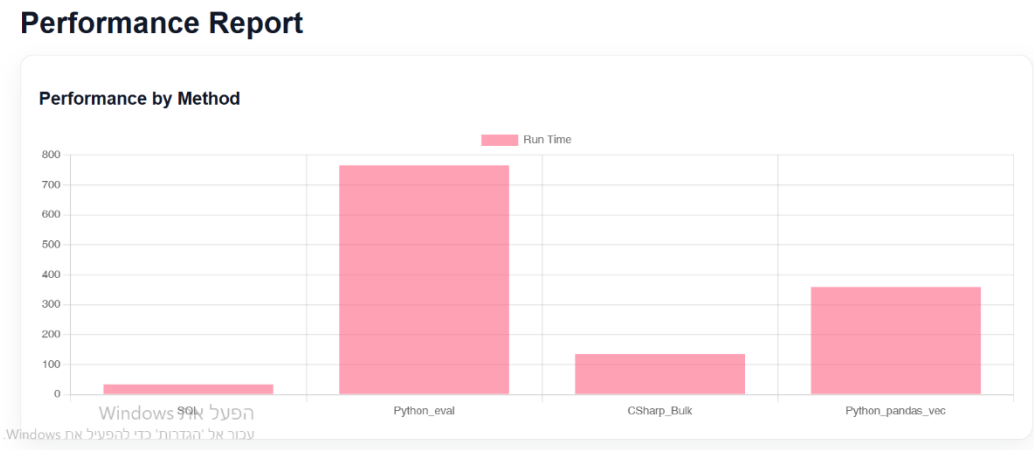
לסיכום, ראיתי כי במערכות עתירות נתונים, מיקום החישוב עשוי להשפיע על הביצועים אף יותר מהאלגוריתם עצמו, ולכן יש לבחור את הפתרון בהתאם לאיזון בין ביצועים, גמישות ותחזוקתיות.

5. הדמיית המערכת(Dashboard)

במסגרת הפרויקט פיתחתי ממשק משתמש באמצעות Angular , המאפשר צפייה ויזואלית בתוצאות ובביצועי השיטות השונות.
הממשק מציג את הנתונים המתקבלים מה־ API בצורה גרפית וטבלאית, ובכך מאפשר ניתוח ברור ונוח של הביצועים.

הממשק כולל:
- טבלה המציגה את תוצאות הלוגים (Logs) עבור כל שיטה, כולל Run Time, Calc Time ו־ Write Time
- גרף השוואתי המציג את זמני הריצה בין השיטות השונות.

להלן דוגמאות מתוך המערכת:



Logs

TARGIL	METHOD	RUN TIME	CALC TIME	WRITE TIME
1	SQL	4.827	0	0
2	SQL	5.094	0	0
3	SQL	4.759	0	0
4	SQL	4.725	0	0
5	SQL	4.844	0	0
6	SQL	4.874	0	0
7	SQL	4.589	0	0
1	Python_eval	96.96835231781006	21.901631116867065	75.066721200943

הפעל את Windows
עבור אל 'הגדרות' כדי להפעיל אר

הדשבורד מדגיש בצורה ברורה את הפערים בין השיטות ומחזק את המסקנות שעלו מהניתוח המספרי.